

<pre>### Tipos de datos int numeroEntero = 10; byte edad = 36; double precio = 19.99; float altura = 1.80f; // La 'f' indica float. boolean esMayorDeEdad = true; char inicial = 'K'; ### Casting. (int) (long) (float) (char) Integer.parseInt() Double.parseDouble() Clase.toString ### Operadores Aritméticos int suma = a + b; int resta = a - b; int multiplicacion = a * b; int division = a / b; int modulo = a % b ### Operador de Asignación y Compuestos saldo += 50; saldo -= 20; ## Operadores relacionales == // igual != // distinto > // mayor que < // menor que >= // mayor o igual <= // menor o igual ## Operadores lógicos ! // NOT (niega) & // AND // OR ^ // XOR (solo uno true) && // AND // OR ### Operador Ternario (condición) ? exp1 : exp2</pre>	<pre>### Condición IF/ELSE. if (expresión-lógica) { sentencias; } else { sentencias; ### Condición SWITCH. switch (valor) { case valor1: sentencias; break; case valorN: sentencias; break; default: sentencias-default; } ### Bucle While while (condición) { sentencias; } ### Bucle Do-While do { sentencias; } while (condición); ### Bucle For for (variableInicializacion; condicion; incremento++) { sentencias. } # Excepciones. try { // Código que puede lanzar una excepción } catch (TipoExcepcion1 e) { // Control excepción 1 } catch (TipoExcepcion2 e) { // Control excepción 2 } finally { // Siempre se ejecuta (haya excepción o no) } Lanzar una nueva excepción delegada throw new Exception("Error bla bla");</pre>	<pre>### Clase String Convierte a String. String.valueOf (); Devuelve la longitud String.length(); Comprueba si está vacía String.isEmpty(); Obtiene el carácter en una posición String.charAt(0); Extrae una letra de la poscion X Y String.substring(2, 6); Convierte a mayúsculas String.toUpperCase(); Convierte a minúsculas String.toLowerCase(); Elimina espacios al principio y al final String.trim(); Comprueba si contiene una subcadena String.contains("Mundo"); // Comparar cadenas String.equals(b); String.equalsIgnoreCase(b); ### Clase Math // Valor absoluto (+) Math.abs(valor); // Redondeo al entero más cercano Math.round(valor); // Redondeo hacia arriba Math.ceil(valor); // Redondeo hacia abajo Math.floor(valor); // X elevado a Y Math.pow(X, Y); // Raíz cuadrada N Math.sqrt(N); // El valor máximo Math.max(a, y); // El valor minimo Math.min(10, 20); // Número aleatorio Math.random();</pre>	<pre>## Paquetes e Importaciones Definir un paquete. package com.miempresa.utilidades; Importa una clase específica para poder usarla import java.util.Scanner; Importa todas las clases de un paquete import java.util.*; ### Objeto Scanner new Scanner(System.in); Lee una línea completa de texto sc.nextLine(); Lee un número entero sc.nextInt(); Cerrar el scanner para liberar recursos sc.close(); Nota: Si vas a leer una línea de texto (`nextLine`) después, es necesario consumir el carácter de nueva línea que queda en el búfer. sc.nextLine(); (Limpia el búfer) ### Salida por Consola Imprime y añade un salto de línea System.out.println("Línea de texto."); Imprime sin salto de línea System.out.print("Texto sin salto."); Imprime con formato (similar a C) System.out.printf("bla bla %s xxx %d %n", var1, var2); %s para strings, %d para enteros, %f para decimales, %n para salto de linea Printf formatear decimales: %f -> 6 decimales %.2f->2 decimales %.3f->3 decimales ### Clase StringBuilder StringBuilder sb = new StringBuilder(); // Añadir un elemento. sb.append("XX"); // Obtener la longitud. sb.length(); // Borrar un carácter de una posicion. sb.deleteCharAt(5); // Convertir a String sb.toString();</pre>
--	---	--	--

<pre>### Clase HashMap import java.util.HashMap; // Declarar un HashMap. HashMap<TipoClave, TipoValor> mapa; HashMap<String, Libro> MyMapaHash; mapa = new HashMap<TipoClave, TipoValor>(); // Añadir un par clave-valor: mapa.put(clave, valor) // Obtener un valor mediante clave: mapa.get(clave) // Comprobar si existe una clave: mapa.containsKey(clave) // Comprobar si existe un valor: mapa.containsValue(valor) // Eliminar por clave: mapa.remove(clave) // Número de elementos: mapa.size(); // Comprobar si está vacío: mapa.isEmpty(); // Recorrer claves. for (TipoClave clave : mapa.keySet()) // Recorrer valores. for (TipoValor valor : mapa.values()) ### Clase JAVAX PERSISTENCIA OBJETOS OBJ import javax.persistence.Entity; import javax.persistence.Id; // ENTIDAD persistente @Entity public class objeto { // Clave primaria @Id private String titulo; private String director; } // Crear fábrica de persistencia (.odb) EntityManagerFactory emf = Persistence.createEntityManagerFactory("peliculas.odb"); // Crear gestor de entidades EntityManager em = emf.createEntityManager(); // INICIAR Y GUARDAR TX em.getTransaction().begin(); em.getTransaction().commit(); // INSERTAR: em.persist(objeto); // BUSCAR POR ID: Pelicula p = em.find(Pelicula.class, titulo); // ACTUALIZAR: p.setDuracion(120); // ELIMINAR: em.remove(p); // Obtener lista List<Película> lista = em.createQuery("SELECT p FROM Película p", Película.class).getResultList(); // CERRAR: em.close(); emf.close();</pre>	<pre>### Clase ArrayList import java.util.ArrayList; // Declarar un ArrayList. ArrayList<String> lista; ArrayList<Integer> lista; ArrayList<Tipo> lista; // Declarar y crear a la vez. ArrayList<Tipo> lista = new ArrayList<Tipo>(); // Añadir un elemento. lista.add(elemento); // Obtener un elemento por posición. lista.get(0); // Cambiar un elemento. lista.set(0, elemento); // Borrar un elemento por posición. lista.remove(0); // Borrar un elemento por valor. lista.remove(elemento); // Saber cuántos elementos hay. lista.size(); // Comprobar si está vacía. lista.isEmpty(); // Comprobar si contiene un elemento. lista.contains(elemento); // Recorrer con for-each. for (Tipo item : lista) {} ### Clase File import java.io.File; // Crear un objeto File con una ruta. File fichero = new File("datos.txt"); // Comprobar si existe: fichero.exists(); // Comprobar si es un fichero. fichero.isFile(); // Comprobar si es un directorio. fichero.isDirectory(); // Obtener el nombre. fichero.getName(); // Obtener la ruta. fichero.getPath();</pre>	<pre>### Clase FileReader BufferedReader import java.io.BufferedReader; import java.io.FileReader; // Syntaxis de uso. FileReader fr = new FileReader("myFichero.txt"); BufferedReader br = new BufferedReader(fr); Leer una línea: br.readLine(); // Leer todas las líneas. while ((línea = br.readLine()) != null) {} Cerrar el lector: br.close(); ### Clase BufferedWriter FileWriter import java.io.BufferedWriter; import java.io.FileWriter; // Crear un BufferedWriter a partir de un FileWriter. FileWriter fw = new FileWriter("myFichero.txt"); BufferedWriter bw = new BufferedWriter(fw); Escribir texto: bw.write("Hola"); Insertar salto línea: bw.newLine(); Guardar cambios: bw.flush(); Cerrar el escritor: bw.close(); ### Array // Crear un array con tamaño fijo. String[] nombre = new String[10]; int[] numeros = new int[3]; // Guardar un elemento en una posición. nombre[0] = valor; // Obtener un elemento. nombre[0]; // Longitud del array. nombre.length; // Recorrer con for-each. for (Tipo elemento : nombre)</pre>	<pre>### Clases Pattern y Matcher (Regex en Java) import java.util.regex.Pattern; import java.util.regex.Matcher; // Crear el patrón (regex) Pattern patron = Pattern.compile("regex"); // Crear el comprobador con el texto Matcher test = patron.matcher(texto); // Validar toda la cadena test.matches() // patrones regex // Solo letras (con acentos y espacios): "^[a-zA-ZáéíóúÁÉÍÓÚÑ]+\$" // Letras y números: "^[a-zA-Z0-9]+\$" // Solo números: "^[0-9]+\$" ### JDBC / MySQL / MariaDB String URL = "jdbc:mariadb://localhost:3306/prueba"; // ABRIR CONEXIÓN SQL Connection conn = DriverManager.getConnection(URL, user, password); // INSERT String sql = "INSERT INTO tabla (campo1, campo2) VALUES (?, ?)"; PreparedStatement stmt = conn.prepareStatement(sql); stmt.setString(1, "valor1"); stmt.setInt(2, 10); stmt.executeUpdate(); // UPDATE String sql2 = "UPDATE tabla SET campo = ? WHERE id = ?"; PreparedStatement stmt2 = conn.prepareStatement(sql2); stmt2.setInt(1, 20); stmt2.setString(2, "A1"); stmt2.executeUpdate(); // DELETE String sql3 = "DELETE FROM tabla WHERE id = ?"; PreparedStatement stmt3 = conn.prepareStatement(sql3); stmt3.setString(1, "A1"); stmt3.executeUpdate(); // SELECT String sql4 = "SELECT * FROM tabla"; Statement st = conn.createStatement(); ResultSet rs = st.executeQuery(sql4); while (rs.next()) { rs.getString("campo"); rs.getInt("edad"); } // CERRAR conn.close(); stmt.close(); rs.close();</pre>
---	---	---	--

